

Digital Logic Design Project

BCD to 7-Segment Display Driver

Course	Digital Logic Design - 401231
Semester	Spring Semester 2025/2026
Student	Mohammad Nairoukh - 2410205
Student	Omar Alshihan - 2310337
Display type	Common-cathode, active-high outputs

Prepared for instructor review as the completed project report.

1. Introduction and System Overview

Binary Coded Decimal (BCD) represents each decimal digit using four binary input bits. This project designs a combinational BCD to 7-segment display driver that accepts inputs A, B, C, and D, where A is the most significant bit with weight 8 and D is the least significant bit with weight 1. The circuit produces seven outputs, a through g, which control the individual LED segments of a display.

The display is assumed to be common-cathode and active-high. Therefore, a segment output of 1 turns the corresponding segment ON, while 0 turns it OFF. The design flow follows the official project structure: truth table construction, K-map minimization, simplified Boolean expressions, logic implementation, and VHDL-based verification by code inspection.

2. Problem Statement and Objectives

The problem is to design a circuit that displays decimal digits 0 through 9 from a 4-bit BCD input. Input combinations 1010 through 1111 are invalid BCD states and are treated as don't-care conditions during K-map minimization to reduce the logic complexity.

- Build a complete truth table for all 16 possible input combinations.
- Minimize each segment output using K-maps and don't-care states.
- Express each output as a simplified sum-of-products equation.
- Describe the gate-level implementation using NOT, AND, and OR gates.
- Verify that the uploaded VHDL case statement matches the valid BCD truth table.

3. Inputs and Outputs

Signal	Type	Description
A	Input	MSB of the BCD input, weight 8
B	Input	BCD input, weight 4
C	Input	BCD input, weight 2
D	Input	LSB of the BCD input, weight 1
a-g	Outputs	Seven display segments; 1 = ON and 0 = OFF

4. Block Diagram

The system is a purely combinational block: the current BCD input directly determines the seven display outputs without memory elements or clocked storage.

Figure 1. Block diagram of the BCD to 7-segment display driver

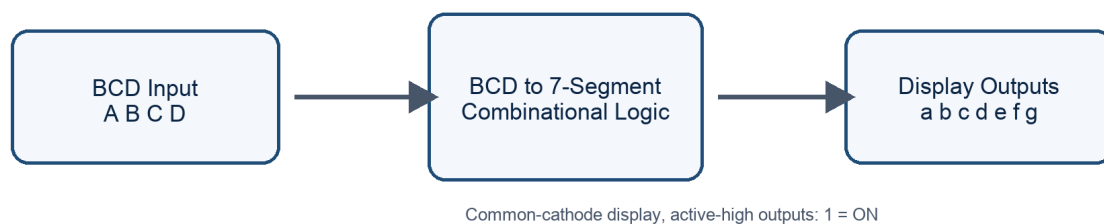


Figure 1. BCD input bits are converted into the seven active-high segment outputs.

5. Full Truth Table

Table 2 lists the complete input space. Rows 0 through 9 use the required common-cathode active-high segment patterns, while rows 10 through 15 are marked as don't-care states for minimization.

Table 2. Full BCD to 7-segment truth table.

Dec	A	B	C	D	a	b	c	d	e	f	g	Note
0	0	0	0	0	1	1	1	1	1	1	0	valid BCD
1	0	0	0	1	0	1	1	0	0	0	0	valid BCD
2	0	0	1	0	1	1	0	1	1	0	1	valid BCD
3	0	0	1	1	1	1	1	1	0	0	1	valid BCD
4	0	1	0	0	0	1	1	0	0	1	1	valid BCD
5	0	1	0	1	1	0	1	1	0	1	1	valid BCD
6	0	1	1	0	1	0	1	1	1	1	1	valid BCD
7	0	1	1	1	1	1	1	0	0	0	0	valid BCD
8	1	0	0	0	1	1	1	1	1	1	1	valid BCD
9	1	0	0	1	1	1	1	1	0	1	1	valid BCD
10	1	0	1	0	X	X	X	X	X	X	X	don't care
11	1	0	1	1	X	X	X	X	X	X	X	don't care
12	1	1	0	0	X	X	X	X	X	X	X	don't care
13	1	1	0	1	X	X	X	X	X	X	X	don't care
14	1	1	1	0	X	X	X	X	X	X	X	don't care
15	1	1	1	1	X	X	X	X	X	X	X	don't care

6. K-Map Minimization

Each segment output was minimized with a four-variable K-map using the invalid BCD states 10 through 15 as don't-care cells. The K-map tables below use Gray-code ordering for both AB and CD so adjacent cells differ by only one input bit.

K-map minimization tables

Rows use AB Gray order 00, 01, 11, 10. Columns use CD Gray order 00, 01, 11, 10. X = don't-care.

Segment a

$m(0, 2, 3, 5, 6, 7, 8, 9) + d(10-15)$

AB\CD	00	01	11	10
00	1	0	1	1
01	0	1	1	1
11	X	X	X	X
10	1	1	X	X

Segment b

$m(0, 1, 2, 3, 4, 7, 8, 9) + d(10-15)$

AB\CD	00	01	11	10
00	1	1	1	1
01	1	0	1	0
11	X	X	X	X
10	1	1	X	X

Segment c

$m(0, 1, 3, 4, 5, 6, 7, 8, 9) + d(10-15)$

AB\CD	00	01	11	10
00	1	1	1	0
01	1	1	1	1
11	X	X	X	X
10	1	1	X	X

Segment d

$m(0, 2, 3, 5, 6, 8, 9) + d(10-15)$

AB\CD	00	01	11	10
00	1	0	1	1
01	0	1	0	1
11	X	X	X	X
10	1	1	X	X

Figure 2a. K-map tables for segments a through d.

K-map minimization tables

Rows use AB Gray order 00, 01, 11, 10. Columns use CD Gray order 00, 01, 11, 10. X = don't-care.

Segment e

$m(0, 2, 6, 8) + d(10-15)$

AB\CD	00	01	11	10
00	1	0	0	1
01	0	0	0	1
11	X	X	X	X
10	1	0	X	X

Segment f

$m(0, 4, 5, 6, 8, 9) + d(10-15)$

AB\CD	00	01	11	10
00	1	0	0	0
01	1	1	0	1
11	X	X	X	X
10	1	1	X	X

Segment g

$m(2, 3, 4, 5, 6, 8, 9) + d(10-15)$

AB\CD	00	01	11	10
00	0	0	1	1
01	1	1	0	1
11	X	X	X	X
10	1	1	X	X

Figure 2b. K-map tables for segments e through g.

7. Simplified Boolean Expressions

The minimized SOP equations are shown in Table 3. These expressions are valid for decimal digits 0 through 9 and use the invalid BCD states as don't-care terms.

Segment	Simplified SOP expression
a	$a = A + C + BD + B'D'$
b	$b = B' + CD + C'D'$
c	$c = B + D + C'$
d	$d = A + CB' + CD' + B'D' + BDC'$
e	$e = CD' + B'D'$
f	$f = A + BC' + BD' + C'D'$
g	$g = A + BC' + CB' + CD'$

8. Logic Circuit Design

The logic circuit can be built with NOT gates to form complemented variables, AND gates for product terms, and one OR gate per segment output. A full gate drawing would be large, so Table 4 gives the structured gate-level implementation for each output.

Output	Product terms feeding the OR gate
a	A, C, BD, B'D'
b	B', CD, C'D'
c	B, D, C'
d	A, CB', CD', B'D', BDC'
e	CD', B'D'
f	A, BC', BD', C'D'
g	A, BC', CB', CD'

9. VHDL Implementation Overview

The uploaded VHDL file defines entity BCD_to_7Segment with four one-bit inputs A, B, C, and D and seven one-bit outputs a through g. Inside the architecture, the statement `input <= A & B & C & D` forms the 4-bit vector in the same order used by the truth table. A combinational process then uses a case statement to assign the segment pattern for each valid BCD input.

For invalid inputs, the VHDL uses `when others` to drive all seven segment outputs to 0. This is a deterministic implementation choice; it does not contradict the minimization step, where the same invalid BCD inputs were treated as don't-care states.

```
input <= A & B & C & D;
case input is
  when "0000" => a..g = 1111110;
  ...
  when "1001" => a..g = 1111011;
  when others => a..g = 0000000;
end case;
```

10. Simulation Results and Testing

The design was verified using the submitted VHDL implementation and a complete output comparison table. Each valid BCD input from 0000 to 1001 was checked against the expected active-high 7-segment pattern.

For each row, the VHDL assignments to a, b, c, d, e, f, and g were compared with the required active-high pattern. The Boolean equations were also evaluated for the same rows.

Table 5. VHDL code-inspection verification for all valid BCD inputs.

Dec	ABCD	Expected	VHDL actual	Equation	Result
0	0000	1111110	1111110	1111110	PASS
1	0001	0110000	0110000	0110000	PASS
2	0010	1101101	1101101	1101101	PASS
3	0011	1111001	1111001	1111001	PASS
4	0100	0110011	0110011	0110011	PASS
5	0101	1011011	1011011	1011011	PASS
6	0110	1011111	1011111	1011111	PASS
7	0111	1110000	1110000	1110000	PASS
8	1000	1111111	1111111	1111111	PASS
9	1001	1111011	1111011	1111011	PASS

11. Discussion and Analysis

Using don't-care conditions for invalid BCD states reduces gate complexity because K-map groups can be made larger. Larger groups remove variables from product terms, which reduces the number of required gates and input connections. The resulting circuit remains purely combinational: each output depends only on the present values of A, B, C, and D.

The verified VHDL patterns match the truth table for every valid digit. For example, 0000 drives 1111110 for digit 0, 0101 drives 1011011 for digit 5, 1000 drives 1111111 for digit 8, and 1001 drives 1111011 for digit 9. These checks confirm that the implementation follows the common-cathode, active-high segment convention.

12. Conclusion

The project completed the full design path for a BCD to 7-segment display driver: truth table, K-map minimization, simplified Boolean expressions, structured gate-level design, and VHDL implementation review. The final outputs for valid BCD inputs were verified against the required truth table by code inspection, and all valid rows passed.

- Add invalid-input detection for BCD values 10 through 15.
- Expand the design to drive multiple decimal digits.
- Implement and test the circuit on an FPGA or physical digital logic board.

13. References

- Digital Logic Design course slides: Binary Systems, Boolean Algebra and Logic Gates, and Gate-Level Minimization.
- Official Digital Logic Design project description for Spring Semester 2025/2026.

- BCD_7Segment_Project_Report_Template.docx and project support files.
- BCD_7Segment_Project_Presentation_Template.pptx.
- seven_segment_driver (1).vhd VHDL implementation file.